



LEARNER GUIDE

AI Vibe Coding for Full Stack Web Development

C1800 · v12.0 · 14 September 2026 · UEN 201200696W

Document Version Control Record

Version	Date	Change
11.0	Legacy	Vue source reference
12.0	14 September 2026	React full-stack methods, AI review gates, ten labs

Contents

Course information · five topics · lab guides

Course method

For every feature, specify the user-visible acceptance test, choose boundaries and data ownership, ask AI for a bounded change, inspect the diff, run tests, then verify the live behavior. Human decisions cover API design, data lifecycle, security, reliability and production operations.

Prerequisites

Install Node.js LTS, npm, VS Code and a modern browser. Use an AI coding assistant only with project data that may be shared. Never paste live credentials into a prompt.

Lab 1: Requirements, architecture and AI prompt contract

Folder: labs/lab-01-requirements-and-architecture. Topic 1; LO1.

1. Open labs/lab-01-requirements-and-architecture and inspect mock-data/tasks.json.
2. Run `npm install` and `npm run dev`; open the local Vite URL.
3. Locate the React root, App component, state, event handler and derived task count.
4. Implement the lab objective: Write a contract, inspect a component tree and verify fixture shape.
5. Write a Given/When/Then acceptance statement for task creation. In src/App.jsx trace fixture → React state → rendered card. Draw the UI → API → data ownership boundary in your evidence log.
6. Run `npm test` and `npm run build`. Fix any failed assertion or build error.
7. Resize to 375 px and desktop width. Confirm controls remain usable and no horizontal scroll appears.
8. Ask an AI assistant for one bounded improvement; compare its diff with the acceptance checks before accepting it.
9. Record the UI screenshot, command output and one design or security trade-off in your evidence log.

Acceptance: the lab README test cases pass and the learner can explain the design and review decision.

Lab 2: React scaffold and responsive shell

Folder: labs/lab-02-react-scaffold. Topic 1; LO1.

1. Open labs/lab-02-react-scaffold and inspect mock-data/tasks.json.
2. Run `npm install` and `npm run dev`; open the local Vite URL.
3. Locate the React root, App component, state, event handler and derived task count.
4. Implement the lab objective: Run a React app and identify the root, entry point and component.
5. In src/main.jsx identify createRoot and #root. Change the page heading, rebuild, and explain why src/App.jsx is the component entry. Confirm the browser console has no mount error.
6. Run `npm test` and `npm run build`. Fix any failed assertion or build error.
7. Resize to 375 px and desktop width. Confirm controls remain usable and no horizontal scroll appears.
8. Ask an AI assistant for one bounded improvement; compare its diff with the acceptance checks before accepting it.
9. Record the UI screenshot, command output and one design or security trade-off in your evidence log.

Acceptance: the lab README test cases pass and the learner can explain the design and review decision.

Lab 3: Responsive task board

Folder: labs/lab-03-responsive-task-board. Topic 2; LO2.

1. Open labs/lab-03-responsive-task-board and inspect mock-data/tasks.json.
2. Run `npm install` and `npm run dev`; open the local Vite URL.
3. Locate the React root, App component, state, event handler and derived task count.
4. Implement the lab objective: Inspect card behavior at 375 px and desktop width.
5. Inspect src/style.css grid-template-columns. Set the viewport to 375 px and 1280 px; record card columns and check for horizontal overflow. Keep `min(100%,16rem)` to avoid a mobile-width card overflow.
6. Run `npm test` and `npm run build`. Fix any failed assertion or build error.
7. Resize to 375 px and desktop width. Confirm controls remain usable and no horizontal scroll appears.
8. Ask an AI assistant for one bounded improvement; compare its diff with the acceptance checks before accepting it.

9. Record the UI screenshot, command output and one design or security trade-off in your evidence log.

Acceptance: the lab README test cases pass and the learner can explain the design and review decision.

Lab 4: State, controls and derived views

Folder: labs/lab-04-state-and-controls. Topic 2; LO2.

1. Open labs/lab-04-state-and-controls and inspect mock-data/tasks.json.
2. Run `npm install` and `npm run dev`; open the local Vite URL.
3. Locate the React root, App component, state, event handler and derived task count.
4. Implement the lab objective: Toggle status and derive the open count from state.
5. In src/App.jsx toggle t-101 and confirm openCount changes from two to one. Explain why the count is derived from tasks rather than stored separately. Repeat after adding a task.
6. Run `npm test` and `npm run build`. Fix any failed assertion or build error.
7. Resize to 375 px and desktop width. Confirm controls remain usable and no horizontal scroll appears.
8. Ask an AI assistant for one bounded improvement; compare its diff with the acceptance checks before accepting it.
9. Record the UI screenshot, command output and one design or security trade-off in your evidence log.

Acceptance: the lab README test cases pass and the learner can explain the design and review decision.

Lab 5: Forms and validation

Folder: labs/lab-05-forms-and-validation. Topic 3; LO3.

1. Open labs/lab-05-forms-and-validation and inspect mock-data/tasks.json.
2. Run `npm install` and `npm run dev`; open the local Vite URL.
3. Locate the React root, App component, state, event handler and derived task count.
4. Implement the lab objective: Reject an empty title, add a valid task and prevent duplicate submit.

5. Submit an empty title and verify role=alert. Submit a valid title and verify a new stable id. Check the trimmed 120-character boundary and describe why the API must repeat validation.
6. Run `npm test` and `npm run build`. Fix any failed assertion or build error.
7. Resize to 375 px and desktop width. Confirm controls remain usable and no horizontal scroll appears.
8. Ask an AI assistant for one bounded improvement; compare its diff with the acceptance checks before accepting it.
9. Record the UI screenshot, command output and one design or security trade-off in your evidence log.

Acceptance: the lab README test cases pass and the learner can explain the design and review decision.

Lab 6: Search, asynchronous requests and races

Folder: labs/lab-06-search-and-async. Topic 3; LO3.

1. Open labs/lab-06-search-and-async and inspect mock-data/tasks.json.
2. Run `npm install` and `npm run dev`; open the local Vite URL.
3. Locate the React root, App component, state, event handler and derived task count.
4. Implement the lab objective: Filter tasks and explain how an obsolete response is discarded.
5. Type a search term and verify only matching tasks display. Add an empty-result message. Sketch an AbortController cleanup for a remote search to prevent an old response replacing a new query.
6. Run `npm test` and `npm run build`. Fix any failed assertion or build error.
7. Resize to 375 px and desktop width. Confirm controls remain usable and no horizontal scroll appears.
8. Ask an AI assistant for one bounded improvement; compare its diff with the acceptance checks before accepting it.
9. Record the UI screenshot, command output and one design or security trade-off in your evidence log.

Acceptance: the lab README test cases pass and the learner can explain the design and review decision.

Lab 7: Reusable components and prop contracts

Folder: labs/lab-07-components-and-props. Topic 4; LO4.

1. Open labs/lab-07-components-and-props and inspect mock-data/tasks.json.
2. Run `npm install` and `npm run dev`; open the local Vite URL.
3. Locate the React root, App component, state, event handler and derived task count.
4. Implement the lab objective: Move task display into a prop-driven child component.
5. Locate TaskCard in src/App.jsx. Confirm task and onToggle are props and the parent owns tasks. Move TaskCard to src/TaskCard.jsx without changing behavior; preserve `key={task.id}`.
6. Run `npm test` and `npm run build`. Fix any failed assertion or build error.
7. Resize to 375 px and desktop width. Confirm controls remain usable and no horizontal scroll appears.
8. Ask an AI assistant for one bounded improvement; compare its diff with the acceptance checks before accepting it.
9. Record the UI screenshot, command output and one design or security trade-off in your evidence log.

Acceptance: the lab README test cases pass and the learner can explain the design and review decision.

Lab 8: Express API and status contracts

Folder: labs/lab-08-express-api. Topic 4; LO4.

1. Open labs/lab-08-express-api and inspect mock-data/tasks.json.
2. Run `npm install` and `npm run dev`; open the local Vite URL.
3. Locate the React root, App component, state, event handler and derived task count.
4. In a second terminal, run `npm run api`; use curl on GET /api/tasks and POST /api/tasks with JSON. Confirm 200, 201 and 400 responses.
5. Implement the lab objective: Request tasks and create a resource with correct HTTP status.
6. Start npm run api. Use curl `http://localhost:3001/api/tasks`, then `curl -X POST -H "Content-Type: application/json" -d '{"title":"Review API"}' http://localhost:3001/api/tasks`. Compare 200, 201 and 400 cases.

7. Run ``npm test`` and ``npm run build``. Fix any failed assertion or build error.
8. Resize to 375 px and desktop width. Confirm controls remain usable and no horizontal scroll appears.
9. Ask an AI assistant for one bounded improvement; compare its diff with the acceptance checks before accepting it.
10. Record the UI screenshot, command output and one design or security trade-off in your evidence log.

Acceptance: the lab README test cases pass and the learner can explain the design and review decision.

Lab 9: SQLite persistence and query safety

Folder: labs/lab-09-sqlite-persistence. Topic 5; LO5.

1. Open labs/lab-09-sqlite-persistence and inspect mock-data/tasks.json.
2. Run ``npm install`` and ``npm run dev``; open the local Vite URL.
3. Locate the React root, App component, state, event handler and derived task count.
4. In a second terminal, run ``npm run api``; use curl on GET /api/tasks and POST /api/tasks with JSON. Confirm 200, 201 and 400 responses.
5. Implement the lab objective: Restart the API and verify a created task survives.
6. Start npm run api and create a task via POST. Stop and restart the API; GET must still contain its id. Inspect server/index.js parameterized INSERT and owner-scoped SELECT. Delete tasks.sqlite only to reset this classroom fixture.
7. Run ``npm test`` and ``npm run build``. Fix any failed assertion or build error.
8. Resize to 375 px and desktop width. Confirm controls remain usable and no horizontal scroll appears.
9. Ask an AI assistant for one bounded improvement; compare its diff with the acceptance checks before accepting it.
10. Record the UI screenshot, command output and one design or security trade-off in your evidence log.

Acceptance: the lab README test cases pass and the learner can explain the design and review decision.

Lab 10: Security, tests and release verification

Folder: labs/lab-10-security-tests-and-release. Topic 5; LO5.

1. Open labs/lab-10-security-tests-and-release and inspect mock-data/tasks.json.
2. Run `npm install` and `npm run dev`; open the local Vite URL.
3. Locate the React root, App component, state, event handler and derived task count.
4. In a second terminal, run `npm run api`; use curl on GET /api/tasks and POST /api/tasks with JSON. Confirm 200, 201 and 400 responses.
5. Implement the lab objective: Reject bad input, run tests and record the deployment smoke check.
6. Run invalid POST and PATCH cases and record status codes. Attempt a guessed id and require 404. Run npm test and npm run build; record commit id, live smoke GET/POST result and rollback target before declaring release.
7. Run `npm test` and `npm run build`. Fix any failed assertion or build error.
8. Resize to 375 px and desktop width. Confirm controls remain usable and no horizontal scroll appears.
9. Ask an AI assistant for one bounded improvement; compare its diff with the acceptance checks before accepting it.
10. Record the UI screenshot, command output and one design or security trade-off in your evidence log.

Acceptance: the lab README test cases pass and the learner can explain the design and review decision.